```
!pip uninstall -y pandas
!pip install pandas==2.1.4
```

```
Found existing installation: pandas 2.2.2
Uninstalling pandas-2.2.2:
  Successfully uninstalled pandas-2.2.2
Collecting pandas==2.1.4
  Downloading pandas-2.1.4-cp312-cp312-manylinux_2_17_x86_64.manylinux2
Collecting numpy<2,>=1.26.0 (from pandas==2.1.4)
  Downloading numpy-1.26.4-cp312-cp312-manylinux_2_17_x86_64.manylinux2
     ━━━
```

Lightweight RAG Chatbot for College Campus Information

A CPU-optimized Retrieval-Augmented Generation system using open-source tools.

```
#Imports
import json
import os
from typing import List, Dict, Tuple
import warnings
warnings.filterwarnings('ignore')

# Core RAG Components
from sentence_transformers import SentenceTransformer
import faiss
import numpy as np
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
import torch
```

SECTION 1: DATA PREPARATION

```
import json
from typing import List, Dict


class CollegeDataManager:
    """Manages college data loading and preparation."""

    @staticmethod
    def create_sample_data() -> List[Dict]:
        """Create structured college data for MITS."""

        return [{
                "category": "college_info",
                "content": "Madanapalle Institute of Technology & Science (M
              },
              {
                "category": "college_info",
                "content": "MITS is situated on a spacious 26.17-acre campus
              },
              {
                "category": "college_info",
                "content": "MITS was founded under the Ratakonda Ranga Reddy
              },
              {
                "category": "college_info",
                "content": "MITS has been conferred the status of a Deemed t
              },
              {
                "category": "college_info",
                "content": "MITS Deemed to be University is governed by the
              },
              {
                "category": "college_info",
                "content": "With 27 years of academic excellence, MITS has e
              },
              {
```

```
    "category": "college_info",
    "content": "MITS has secured NIRF India Ranking 2025 in the
  },
  {
    "category": "college_info",
    "content": "MITS Deemed to be University has been honored wi
  },
  {
    "category": "college_info",
    "content": "The institution's vision and mission is to serve
  },
  {
    "category": "college_info",
    "content": "MITS has global collaborations and MOUs with top
  },

  {
    "category": "ug_programs",
    "content": "B.Tech. - Computer Science & Engineering: Core c
  },
  {
    "category": "ug_programs",
    "content": "B.Tech. - CSE (Artificial Intelligence & Data Sc
  },
  {
    "category": "ug_programs",
    "content": "B.Tech. - CSE (Artificial Intelligence & Machine
  },
  {
    "category": "ug_programs",
    "content": "B.Tech. - CSE (Artificial Intelligence & Robotic
  },
  {
    "category": "ug_programs",
    "content": "B.Tech. - CSE (Cyber Security): Learn ethical ha
  },
  {
    "category": "ug_programs",
    "content": "B.Tech. - Electronics & Communication Engineerin
  },
  {
    "category": "ug_programs",
    "content": "B.Tech. - Electrical & Electronics Engineering:
  },
  {
    "category": "ug_programs",
    "content": "B.Tech. - Mechanical Engineering: Design, manufa
  },
  {
    "category": "ug_programs",
    "content": "B.Tech. - Civil Engineering: Learn the fundament
  },
  {
    "category": "ug_programs",
    "content": "B.Tech. - Bioinformatics: Explore computational
  },
  {
    "category": "ug_programs",
    "content": "B.Tech. - Civil & Machine Learning (CiML): An in
  },
  {
    "category": "ug_programs",
    "content": "B.B.A. - Management Studies: Develop strong mana
  },
```

```
{
  "category": "ug_programs",
  "content": "B.C.A. - Computer Applications: A strong foundat
},

{
  "category": "pg_programs",
  "content": "M.Tech. - VLSI & Embedded Systems (ECE): Advance
},
{
  "category": "pg_programs",
  "content": "M.Tech. - Civil Engineering (Construction Engine
},
{
  "category": "pg_programs",
  "content": "M.Tech. - Automation and Robotics (MECH): Advanc
},
{
  "category": "pg_programs",
  "content": "M.Tech. - Electric Vehicle Technology (EEE): Adv
},
{
  "category": "pg_programs",
  "content": "M.Tech. - Computer Science & Engineering: Advanc
},
{
  "category": "pg_programs",
  "content": "M.Tech. - Computer Science & Engineering (AI & M
},
{
  "category": "pg_programs",
  "content": "MBA - Master of Business Administration: Advance
},
{
  "category": "pg_programs",
  "content": "MCA - Master of Computer Applications: Advanced
},

{
  "category": "phd_programs",
  "content": "MITS Deemed to be University offers Ph.D. resear
},
{
  "category": "phd_programs",
  "content": "Ph.D. research programmes are offered in the fol
},
{
  "category": "phd_programs",
  "content": "Ph.D. research programmes are also offered in th
},
{
  "category": "phd_programs",
  "content": "Eligibility for Ph.D. admission: Candidates must
},
{
  "category": "phd_programs",
  "content": "A relaxation of 5% marks is applicable for SC /
},
{
  "category": "phd_programs",
  "content": "Ph.D. selection procedure: Selection is based on
},
{
  "category": "phd_programs",
```

```
    "content": "Ph.D. Important Dates (Revised) for 2025-26 sess
  },
  {
    "category": "phd_programs",
    "content": "For Ph.D. admissions enquiries, contact: deanres
  },

  {
    "category": "admissions",
    "content": "MITS Admissions 2026 are open for UG (B.Tech, BE
  },
  {
    "category": "admissions",
    "content": "Admission Process Step 1 – Registration: Fill th
  },
  {
    "category": "admissions",
    "content": "Admission Process Step 2 – Verification: Verify
  },
  {
    "category": "admissions",
    "content": "Admission Process Step 3 – Fill Application: Upl
  },
  {
    "category": "admissions",
    "content": "Admission Process Step 4 – Entrance Test: Appear
  },
  {
    "category": "admissions",
    "content": "Admission Process Step 5 – Counseling: Attend co
  },
  {
    "category": "admissions",
    "content": "MITSUCET 2026 Key Dates: Last Date to Apply – 31
  },
  {
    "category": "admissions",
    "content": "For admissions, reach MITS at: Phone – 733080355
  },

  {
    "category": "academic_schools",
    "content": "MITS Deemed to be University organizes its acade
  },

  {
    "category": "placements",
    "content": "MITS is committed to 100% placement assistance a
  },
  {
    "category": "placements",
    "content": "Top recruiters include leading companies such as
  },
  {
    "category": "placements",
    "content": "Alumni testimonial – Shabaz Khan .N (B.Tech Civi
  },
  {
    "category": "placements",
    "content": "Alumni testimonial – N. Shariq Nawaz Kh. (B.Tech
  },

  {
    "category": "facilities",
```

```
    "content": "MITS campus spans 26.17 acres along the Madanapa
  },
  {
    "category": "facilities",
    "content": "The campus features newly constructed hostel blo
  },
  {
    "category": "facilities",
    "content": "State-of-the-art laboratories are available acro
  },
  {
    "category": "facilities",
    "content": "The campus provides an equal and inclusive stude
  },

  {
    "category": "events",
    "content": "Ashv-2K26: A Two Day Techno Cultural Fest organi
  },
  {
    "category": "events",
    "content": "Faculty Development Program: A Six-Day Online Fa
  },
  {
    "category": "events",
    "content": "Technical Talk: A One Day Technical Talk on 'Fos
  },

  {
    "category": "accreditations_rankings",
    "content": "MITS holds NAAC A+ accreditation, reflecting its
  },
  {
    "category": "accreditations_rankings",
    "content": "Multiple programmes at MITS are accredited by th
  },
  {
    "category": "accreditations_rankings",
    "content": "MITS is ranked in the NIRF India Rankings 2025 b
  },
  {
    "category": "accreditations_rankings",
    "content": "MITS Deemed to be University is approved by AICT
  },

  {
    "category": "contact",
    "content": "Madanapalle Institute of Technology & Science (M
  },
  {
    "category": "contact",
    "content": "Phone: +91-8571-280255; 280706. Admissions Helpl
  },
  {
    "category": "contact",
    "content": "Email: admissions@mits.ac.in. Official Website:
  },
  {
    "category": "contact",
    "content": "Social Media: Facebook – https://www.facebook.co
  },
  {
    "category": "contact",
    "content": "MITS Radio 90.8 CRS is available for download or
```

```
                }
            ]

        @staticmethod
        def save_to_json(data: List[Dict], filepath: str = "mits_data.json"):
            """Save data to JSON file."""
            with open(filepath, 'w', encoding='utf-8') as f:
                json.dump(data, f, indent=2, ensure_ascii=False)
            print(f"✓ Data saved to {filepath}")

        @staticmethod
        def load_from_json(filepath: str = "mits_data.json") -> List[Dict]:
            """Load data from JSON file."""
            with open(filepath, 'r', encoding='utf-8') as f:
                data = json.load(f)
            print(f"✓ Loaded {len(data)} documents from {filepath}")
            return data
```

```
data = CollegeDataManager.create_sample_data()
print(f"Created {len(data)} records")

    # Step 2: Save to JSO
CollegeDataManager.save_to_json(data)

    # Step 3: Load from JSON
loaded_data = CollegeDataManager.load_from_json()

    # Step 4: Display sample record
print("\nSample record:")
print(loaded_data[0])
```

```
Created 68 records
✓ Data saved to mits_data.json
✓ Loaded 68 documents from mits_data.json

Sample record:
{'category': 'college_info', 'content': 'Madanapalle Institute of Techr
```

SECTION 2: TEXT CHUNKING

```
class TextChunker:
    """Handles document chunking with overlap."""

    def __init__(self, chunk_size: int = 400, overlap: int = 50):
        """
        Initialize chunker.

        Args:
            chunk_size: Maximum tokens per chunk
            overlap: Overlapping tokens between chunks
        """
        self.chunk_size = chunk_size
        self.overlap = overlap
```

```
    def chunk_documents(self, documents: List[Dict]) -> List[Dict]:
        """
        Chunk documents into smaller pieces.

        For this implementation, since our documents are already appropriate
        sized, we'll keep them as-is but add metadata.
        """
        chunks = []
        for idx, doc in enumerate(documents):
            chunks.append({
                "chunk_id": idx,
                "content": doc["content"],
                "category": doc["category"],
                "metadata": {"source": "college_data", "doc_id": idx}
            })

        print(f"✓ Created {len(chunks)} chunks")
        return chunks
```

SECTION 3: EMBEDDING & VECTOR STORE

```
class VectorStore:
    """FAISS-based vector store for semantic search."""

    def __init__(self, embedding_model_name: str = "all-MiniLM-L6-v2"):
        """
        Initialize vector store with embedding model.

        Args:
            embedding_model_name: Sentence transformer model name
        """
        print(f"Loading embedding model: {embedding_model_name}...")
        self.embedder = SentenceTransformer(embedding_model_name)
        self.index = None
        self.chunks = []
        self.dimension = 384  # Dimension for all-MiniLM-L6-v2

    def build_index(self, chunks: List[Dict]):
        """
        Create FAISS index from document chunks.

        Args:
            chunks: List of document chunks with content
        """
        self.chunks = chunks
        texts = [chunk["content"] for chunk in chunks]

        print("Generating embeddings...")
        embeddings = self.embedder.encode(
            texts,
            show_progress_bar=True,
            convert_to_numpy=True
        )

        # Create FAISS index (L2 distance)
        self.index = faiss.IndexFlatL2(self.dimension)
        self.index.add(embeddings.astype('float32'))

        print(f"✓ FAISS index built with {self.index.ntotal} vectors")
```

```
    def search(self, query: str, k: int = 3) -> List[Tuple[Dict, float]]:
        """
        Search for top-k most relevant chunks.

        Args:
            query: User query string
            k: Number of results to return

        Returns:
            List of (chunk, distance) tuples
        """
        if self.index is None:
            raise ValueError("Index not built. Call build_index() first.")

        # Encode query
        query_embedding = self.embedder.encode(
            [query],
            convert_to_numpy=True
        ).astype('float32')

        # Search
        distances, indices = self.index.search(query_embedding, k)

        # Return chunks with scores
        results = []
        for dist, idx in zip(distances[0], indices[0]):
            results.append((self.chunks[idx], float(dist)))

        return results
```

SECTION 4: LLM GENERATOR

```
class LLMGenerator:
    """Lightweight LLM for answer generation."""

    def __init__(self, model_name: str = "google/flan-t5-xl"):
        """
        Initialize LLM generator.

        Args:
            model_name: HuggingFace model identifier
        """
        print(f"Loading LLM: {model_name}...")
        self.tokenizer = AutoTokenizer.from_pretrained(model_name)
        self.model = AutoModelForSeq2SeqLM.from_pretrained(model_name)

        # Use CPU
        self.device = "cuda" if torch.cuda.is_available() else "cpu"
        self.model.to(self.device)
        print(f"✓ Model loaded on {self.device}")

    def generate_answer(
        self,
        query: str,
        context: str,
        max_length: int = 256
    ) -> str:
        """
        Generate answer based on query and retrieved context.
```

```
        Args:
            query: User question
            context: Retrieved relevant context
            max_length: Maximum answer length

        Returns:
            Generated answer string
        """
        # Construct prompt
        prompt = f"""You are a helpful campus assistant.

        Using ONLY the information provided in the context,
        write a complete, well-formed answer in natural language.
        If the answer is not present, say:
        "The information is not available in the provided college data."


    Context: {context}

    Question: {query}

    Answer:"""

        # Tokenize
        inputs = self.tokenizer(
            prompt,
            max_length=512,
            truncation=True,
            return_tensors="pt"
        ).to(self.device)

        # Generate
        outputs = self.model.generate(
                  **inputs,
                  max_length=200,
                  min_length=40,
                  do_sample=False,          # IMPORTANT
                  num_beams=4,
                  repetition_penalty=1.2,   # KEY FIX
                  no_repeat_ngram_size=3,   # HARD STOP repetition
                  early_stopping=True
              )

        # Decode
        answer = self.tokenizer.decode(outputs[0], skip_special_tokens=True)
        return answer.strip()
```

SECTION 5: RAG PIPELINE

```
class CampusRAGChatbot:
    """Complete RAG pipeline for campus information."""

    def __init__(
        self,
        embedding_model: str = "all-MiniLM-L6-v2",
        llm_model: str = "google/flan-t5-xl",
        top_k: int = 3
```

```
    ):
        """
        Initialize RAG chatbot.

        Args:
            embedding_model: Sentence transformer model
            llm_model: LLM for generation
            top_k: Number of chunks to retrieve
        """
        self.vector_store = VectorStore(embedding_model)
        self.generator = LLMGenerator(llm_model)
        self.top_k = top_k

    def setup(self, chunks: List[Dict]):
        """Build vector index from chunks."""
        self.vector_store.build_index(chunks)

    def query(self, question: str, verbose: bool = False) -> Dict:
        """
        Process user query through RAG pipeline.

        Args:
            question: User question
            verbose: Whether to show retrieval details

        Returns:
            Dictionary with answer and metadata
        """
        # Step 1: Retrieve relevant chunks
        results = self.vector_store.search(question, k=self.top_k)

        # Step 2: Prepare context
        context_parts = []
        sources = []

        for chunk, score in results:
            context_parts.append(chunk["content"])
            sources.append({
                "category": chunk["category"],
                "distance": score
            })

        context = "\n\n".join(context_parts)

        if verbose:
            print("\n" + "="*60)
            print("RETRIEVED CONTEXT:")
            print("="*60)
            for i, (chunk, score) in enumerate(results, 1):
                print(f"\n[Chunk {i}] Distance: {score:.4f}")
                print(f"Category: {chunk['category']}")
                print(f"Content: {chunk['content'][:200]}...")
            print("="*60 + "\n")

        # Step 3: Generate answer
        answer = self.generator.generate_answer(question, context)

        return {
            "question": question,
            "answer": answer,
            "sources": sources,
            "num_sources": len(sources)
        }
```

```
    def chat(self):
        """Interactive chat interface."""
        print("\n" + "="*60)
        print("🎓 Ekdant Institute of Technology - Campus Assistant")
        print("="*60)
        print("\nAsk me anything about the college!")
        print("Type 'quit' or 'exit' to end the conversation.\n")

        while True:
            try:
                question = input("You: ").strip()

                if question.lower() in ['quit', 'exit', 'q']:
                    print("\nThank you for using the campus assistant. Goodb
                    break

                if not question:
                    continue

                print("\n🤖 Assistant: ", end="", flush=True)
                result = self.query(question)
                print(result["answer"])
                print()

            except KeyboardInterrupt:
                print("\n\nGoodbye! 👋\n")
                break
            except Exception as e:
                print(f"\n❌ Error: {e}\n")
```

SECTION 6: MAIN EXECUTION

```
def main():
    """Main execution function."""

    print("\n" + "="*70)
    print(" CAMPUS RAG CHATBOT - INITIALIZATION")
    print("="*70 + "\n")

    # Step 1: Prepare data
    print("Step 1: Preparing college data...")
    data_manager = CollegeDataManager()
    college_data = data_manager.create_sample_data()
    data_manager.save_to_json(college_data)

    # Step 2: Chunk documents
    print("\nStep 2: Chunking documents...")
    chunker = TextChunker(chunk_size=400, overlap=50)
    chunks = chunker.chunk_documents(college_data)

    # Step 3: Initialize RAG chatbot
    print("\nStep 3: Initializing RAG pipeline...")
    chatbot = CampusRAGChatbot(
        embedding_model="all-MiniLM-L6-v2",
        llm_model="google/flan-t5-xl",
        top_k=3
    )

    # Step 4: Build vector index
```

```
    print("\nStep 4: Building vector index...")
    chatbot.setup(chunks)

    print("\n" + "="*70)
    print(" INITIALIZATION COMPLETE")
    print("="*70 + "\n")

    # Sample queries for demonstration
    print("Running sample queries...\n")

    sample_queries = [
        "What courses are offered in AI?",
        "What is the attendance policy?",
        "Tell me about TechNova",
        "What facilities are available in the library?",
        "How many seats are there in B.Tech CSE?"
    ]

    for query in sample_queries:
        print(f"\n{'='*70}")
        print(f"Q: {query}")
        print('='*70)
        result = chatbot.query(query, verbose=False)
        print(f"A: {result['answer']}\n")

    # Start interactive chat
    chatbot.chat()


if __name__ == "__main__":
    main()
```

```
======================================================================
 CAMPUS RAG CHATBOT - INITIALIZATION
======================================================================

Step 1: Preparing college data...
✓ Data saved to mits_data.json

Step 2: Chunking documents...
✓ Created 27 chunks

Step 3: Initializing RAG pipeline...
Loading embedding model: all-MiniLM-L6-v2...
modules.json: 100%  349/349 [00:00<00:00, 40.7kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Plea
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthen
config_sentence_transformers.json: 100%  116/116 [00:00<00:00, 11.0kB/s]
README.md: 10.5k/? [00:00<00:00, 960kB/s]
sentence_bert_config.json: 100%  53.0/53.0 [00:00<00:00, 4.65kB/s]
config.json: 100%  612/612 [00:00<00:00, 70.2kB/s]
model.safetensors: 100%  90.9M/90.9M [00:01<00:00, 458MB/s]
Loading weights: 100%  103/103 [00:00<00:00, 409.07it/s, Materializing param=pooler.dense.w

BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                      | Status     |  |
-------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/arch
tokenizer_config.json: 100%  350/350 [00:00<00:00, 36.5kB/s]
vocab.txt: 232k/? [00:00<00:00, 10.9MB/s]
tokenizer.json: 466k/? [00:00<00:00, 22.5MB/s]
special_tokens_map.json: 100%  112/112 [00:00<00:00, 8.55kB/s]
config.json: 100%  190/190 [00:00<00:00, 18.4kB/s]
Loading LLM: google/flan-t5-xl...
config.json: 1.44k/? [00:00<00:00, 135kB/s]
tokenizer_config.json: 2.54k/? [00:00<00:00, 214kB/s]
spiece.model: 100%  792k/792k [00:01<00:00, 3.95MB/s]
tokenizer.json: 2.42M/? [00:00<00:00, 57.3MB/s]
special_tokens_map.json: 2.20k/? [00:00<00:00, 230kB/s]
model.safetensors.index.json: 53.0k/? [00:00<00:00, 3.90MB/s]
```

```
##testing
from sentence_transformers import SentenceTransformer, util
```

```
import numpy as np


class RAGEvaluator:
    def __init__(self, chatbot):
        self.chatbot = chatbot
        self.embedder = SentenceTransformer("all-MiniLM-L6-v2")

    def evaluate(self, dataset):
        results = []

        print("\n" + "="*70)
        print(" RAG EVALUATION STARTED")
        print("="*70 + "\n")

        for i, sample in enumerate(dataset):
            query = sample["query"]
            ground_truth = sample["ground_truth"]

            # Get model answer
            response = self.chatbot.query(query, verbose=False)
            predicted = response["answer"]

            # Compute semantic similarity
            emb1 = self.embedder.encode(predicted, convert_to_tensor=True)
            emb2 = self.embedder.encode(ground_truth, convert_to_tensor=True

            similarity = util.cos_sim(emb1, emb2).item()

            # Exact match (loose)
            exact_match = int(ground_truth.lower() in predicted.lower())

            results.append({
                "query": query,
                "predicted": predicted,
                "ground_truth": ground_truth,
                "similarity": round(similarity, 3),
                "exact_match": exact_match
            })

            print(f"\n[{i+1}] Query: {query}")
            print(f"Predicted: {predicted}")
            print(f"Ground Truth: {ground_truth}")
            print(f"Similarity: {similarity:.3f} | Exact Match: {exact_match

        # Aggregate metrics
        avg_similarity = np.mean([r["similarity"] for r in results])
        accuracy = np.mean([r["exact_match"] for r in results])

        print("\n" + "="*70)
        print(" EVALUATION SUMMARY")
        print("="*70)
        print(f"Average Semantic Similarity: {avg_similarity:.3f}")
        print(f"Exact Match Accuracy: {accuracy:.3f}")

        return results

evaluation_dataset = [
    {
        "category": "College Info",
        "complexity": "Simple",
        "query": "Where is MITS located?",
        "ground_truth": "MITS is located in Madanapalle, Andhra Pradesh, Inc
    },
```

```
    {
        "category": "College Info",
        "complexity": "Technical",
        "query": "What is the status of MITS university?",
        "ground_truth": "MITS has been granted Deemed to be University statu
    },
    {
        "category": "UG Programs",
        "complexity": "Multi-Entity",
        "query": "What CSE specializations are offered?",
        "ground_truth": "CSE specializations include AI and Data Science, AI
    },
    {
        "category": "Admissions",
        "complexity": "Process",
        "query": "What is the admission process for MITS?",
        "ground_truth": "The process includes registration, verification, fi
    },
    {
        "category": "PhD",
        "complexity": "Complex",
        "query": "What is the eligibility for PhD admission?",
        "ground_truth": "Candidates must have a postgraduate degree with at
    },
    {
        "category": "Placements",
        "complexity": "Simple",
        "query": "Which companies recruit from MITS?",
        "ground_truth": "Top recruiters include Cognizant, Infosys, Morgan S
    },
    {
        "category": "Facilities",
        "complexity": "Simple",
        "query": "Does MITS provide hostel facilities?",
        "ground_truth": "Yes, MITS provides hostel facilities for both boys
    },
    {
        "category": "Events",
        "complexity": "Specific",
        "query": "What is Ashv-2K26?",
        "ground_truth": "Ashv-2K26 is a two-day techno cultural fest held at
    },
    {
        "category": "Ranking",
        "complexity": "Technical",
        "query": "What is the NIRF ranking of MITS?",
        "ground_truth": "MITS is ranked in the 201 to 300 band in NIRF 2025.
    },
    {
        "category": "Contact",
        "complexity": "Simple",
        "query": "How can I contact MITS?",
        "ground_truth": "You can contact MITS via phone, email admissions at
    }
]

def main():
    """Main execution function."""

    print("\n" + "="*70)
    print(" CAMPUS RAG CHATBOT - INITIALIZATION")
    print("="*70 + "\n")

    # Step 1: Prepare data
```

```
    print("Step 1: Preparing college data...")
    data_manager = CollegeDataManager()
    college_data = data_manager.create_sample_data()
    data_manager.save_to_json(college_data)

    # Step 2: Chunk documents
    print("\nStep 2: Chunking documents...")
    chunker = TextChunker(chunk_size=400, overlap=50)
    chunks = chunker.chunk_documents(college_data)

    # Step 3: Initialize RAG chatbot
    print("\nStep 3: Initializing RAG pipeline...")
    chatbot = CampusRAGChatbot(
        embedding_model="all-MiniLM-L6-v2",
        llm_model="google/flan-t5-xl",
        top_k=3
    )

    # Step 4: Build vector index
    print("\nStep 4: Building vector index...")
    chatbot.setup(chunks)

    print("\n" + "="*70)
    print(" INITIALIZATION COMPLETE")
    print("="*70 + "\n")

    # Step 5: Run Evaluation
    print("\nStep 5: Running evaluation...\n")

    evaluator = RAGEvaluator(chatbot)
    results = evaluator.evaluate(evaluation_dataset)
if __name__ == "__main__":
    main()
```

```
======================================================================
 CAMPUS RAG CHATBOT - INITIALIZATION
======================================================================

Step 1: Preparing college data...
✓ Data saved to mits_data.json

Step 2: Chunking documents...
✓ Created 68 chunks

Step 3: Initializing RAG pipeline...
Loading embedding model: all-MiniLM-L6-v2...
Loading weights: 100%  103/103 [00:00<00:00, 252.96it/s, Materializing param=pooler.dense.w

BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                      | Status     |  |
-------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/arch
Loading LLM: google/flan-t5-xl...
Loading weights: 100%  558/558 [00:01<00:00, 753.83it/s, Materializing param=shared.weight]

The tied weights mapping and config for this model specifies to tie s
✓ Model loaded on cuda

Step 4: Building vector index...
Generating embeddings...
Batches: 100%                                                3/3 [00:00<00:00,  8.92it/s]
✓ FAISS index built with 68 vectors

======================================================================
 INITIALIZATION COMPLETE
======================================================================


Step 5: Running evaluation...

Loading weights: 100%  103/103 [00:00<00:00, 389.40it/s, Materializing param=pooler.dense.w

BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                      | Status     |  |
-------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/arch

======================================================================
 RAG EVALUATION STARTED
======================================================================
```

```
######
######
```

```
###### STOP RIGHT HEREEEEEEEEEEEEEEEEE
######

[2] Query: What is the status of MITS university?
```

```
# ==========================================
# Initialize RAG Pipeline for Evaluation
# ==========================================

# Step 1: Prepare dataset
data_manager = CollegeDataManager()
college_data = data_manager.create_sample_data()

# Step 2: Chunk documents
chunker = TextChunker(chunk_size=400, overlap=50)
chunks = chunker.chunk_documents(college_data)

# Step 3: Initialize chatbot
chatbot = CampusRAGChatbot(
    embedding_model="all-MiniLM-L6-v2",
    llm_model="google/flan-t5-xl",
    top_k=3
)

# Step 4: Build vector index
chatbot.setup(chunks)

print("RAG Chatbot Ready for Evaluation")
```

```
✓ Created 68 chunks
Loading embedding model: all-MiniLM-L6-v2...
Loading weights: 100%|103/103 [00:00<00:00, 512.56it/s, Materializing param=pooler.dense.we
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                     | Status     |  |
-----------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |
Notes:
- UNEXPECTED    :can be ignored when loading from different task/archit
Loading LLM: google/flan-t5-xl...
Loading weights: 100%|558/558 [00:00<00:00, 739.31it/s, Materializing param=shared.weight]
The tied weights mapping and config for this model specifies to tie sha
✓ Model loaded on cuda
Generating embeddings...
Batches: 100%|3/3 [00:00<00:00, 8.80it/s]
✓ FAISS index built with 68 vectors
RAG Chatbot Ready for Evaluation
```

```
# ==========================================
# Evaluation Dataset for Cross Validation (MITS)
# ==========================================

evaluation_dataset = [
    # =========================
    # Admissions
    # =========================
```

[7] Query: Does MITS provide hostel facilities?
Predicted: Yes, the campus features nearby comfortable hostel blocks.
Ground Truth: Yes, MITS provides hostel facilities for both boys and
Similarity: 0.589 | Exact Match: 0

[8] Query: What is Ashv-2K26?
Predicted: A Two Day Techno Cultural Fest organized at MITS Deemed to
Ground Truth: Ashv-2K26 is a two-day techno cultural fest held at MIT
Similarity: 0.744 | Exact Match: 0

[9] Query: What is the NIRF ranking of MITS?
Predicted: 201-300 under the National Institutional Framework, Minist
Ground Truth: MITS is ranked in the 201 to 300 band in NIRF 2025.
Similarity: 0.558 | Exact Match: 0

[10] Query: How can I contact MITS?
Predicted: Phone: 7330803555/7330853555, Whatsapp: 07330803555 and e
Ground Truth: You can contact MITS via phone, email admissions at mit
Similarity: 0.030 | Exact Match: 0

```
==================================================================
 EVALUATION SUMMARY
```

Exact Match Accuracy: 0.000

```
{
    "category": "Admissions",
    "complexity": "Simple",
    "query": "How can I apply for admission at MITS?",
    "ground_truth": "Students must register online, verify details, fill
},
{
    "category": "Admissions",
    "complexity": "Technical",
    "query": "What is MITSUCET?",
    "ground_truth": "MITSUCET is the MITS University Common Entrance Tes
},
{
    "category": "Admissions",
    "complexity": "Specific",
    "query": "What is the last date to apply for MITSUCET 2026?",
    "ground_truth": "The last date to apply for MITSUCET 2026 is 31 Marc
},

# ==========================
# College Info
# ==========================
{
    "category": "College Info",
    "complexity": "Simple",
    "query": "Where is MITS located?",
    "ground_truth": "MITS is located in Madanapalle, Andhra Pradesh, Inc
},
{
    "category": "College Info",
    "complexity": "Complex",
    "query": "What is the status of MITS as a university?",
    "ground_truth": "MITS has been granted Deemed to be University statu
},
{
    "category": "College Info",
    "complexity": "Technical",
    "query": "What accreditations does MITS have?",
    "ground_truth": "MITS has NAAC A plus accreditation and NBA recognit
},

# ==========================
# Programs (UG / PG)
# ==========================
{
    "category": "Programs",
    "complexity": "Multi-Entity",
    "query": "What CSE specializations are available in MITS?",
    "ground_truth": "CSE specializations include AI and Data Science, AI
},
{
    "category": "Programs",
    "complexity": "Simple",
    "query": "Does MITS offer MBA and MCA programs?",
    "ground_truth": "Yes, MITS offers MBA and MCA postgraduate programs.
},
{
    "category": "Programs",
    "complexity": "Complex",
    "query": "What is the focus of the Bioinformatics program?",
    "ground_truth": "The Bioinformatics program focuses on computational
},

# ==========================
```

```
# PhD Programs
# ==========================
{
    "category": "PhD",
    "complexity": "Technical",
    "query": "What is the eligibility for PhD admission?",
    "ground_truth": "Candidates need a postgraduate degree with at least
},
{
    "category": "PhD",
    "complexity": "Complex",
    "query": "What is the selection process for PhD at MITS?",
    "ground_truth": "Selection involves an entrance test MITS-DAT and ar
},

# ==========================
# Facilities
# ==========================
{
    "category": "Facilities",
    "complexity": "Simple",
    "query": "Does MITS provide hostel facilities?",
    "ground_truth": "Yes, hostel facilities are available for both boys
},
{
    "category": "Facilities",
    "complexity": "Multi-Entity",
    "query": "What facilities are available on campus?",
    "ground_truth": "Facilities include hostels, laboratories, Wi-Fi, sp
},

# ==========================
# Placements
# ==========================
{
    "category": "Placements",
    "complexity": "Simple",
    "query": "Does MITS provide placement assistance?",
    "ground_truth": "Yes, MITS provides 100 percent placement assistance
},
{
    "category": "Placements",
    "complexity": "Multi-Entity",
    "query": "Which companies recruit students from MITS?",
    "ground_truth": "Top recruiters include Cognizant, Infosys, Morgan S
},

# ==========================
# Events
# ==========================
{
    "category": "Events",
    "complexity": "Specific",
    "query": "What is Ashv-2K26?",
    "ground_truth": "Ashv-2K26 is a two-day techno cultural fest held at
},
{
    "category": "Events",
    "complexity": "Complex",
    "query": "What academic events are conducted at MITS?",
    "ground_truth": "MITS conducts workshops, faculty development progra
},

# ==========================
```

```
    # Rankings & Recognition
    # ========================
    {
        "category": "Ranking",
        "complexity": "Technical",
        "query": "What is the NIRF ranking of MITS?",
        "ground_truth": "MITS is ranked in the 201 to 300 band in NIRF 2025.
    },

    # ========================
    # Contact
    # ========================
    {
        "category": "Contact",
        "complexity": "Simple",
        "query": "How can I contact MITS admissions?",
        "ground_truth": "You can contact admissions via phone, email admissi
    }
]

print("Cross Validation Dataset Size:", len(evaluation_dataset))
```

```
Cross Validation Dataset Size: 19
```

```
Loading weights: 100%  103/103 [00:00<00:00, 412.58it/s, Materializing param=pooler.dense.wei

BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                      | Status     |  |
-------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/archit
```

```
# ==========================================
# Correct RAG Evaluation
# ==========================================

import time
import numpy as np
import pandas as pd
from nltk.translate.bleu_score import sentence_bleu, SmoothingFunction

smooth = SmoothingFunction().method1

results = []

for item in evaluation_dataset:

    query = item["query"]
    ground_truth = item["ground_truth"]

    start = time.time()
```

```
        response = chatbot.query(query)
        latency = time.time() - start

        answer = response["answer"]

        # CORRECT CONTEXT EXTRACTION
        retrieved_context = " ".join(
            [chunk["content"] for chunk, score in chatbot.vector_store.search(
        )

        # embeddings
        embeddings = semantic_model.encode([answer, ground_truth, retrieved_co

        ans_vec = embeddings[0]
        gt_vec = embeddings[1]
        ctx_vec = embeddings[2]

        # cosine similarity
        relevance = np.dot(ans_vec, gt_vec) / (
            np.linalg.norm(ans_vec) * np.linalg.norm(gt_vec)
        )

        faithfulness = np.dot(ans_vec, ctx_vec) / (
            np.linalg.norm(ans_vec) * np.linalg.norm(ctx_vec)
        )

        bleu = sentence_bleu(
            [ground_truth.split()],
            answer.split(),
            smoothing_function=smooth
        )

        results.append({
            "Test Category": item["category"],
            "Query Complexity": item["complexity"],
            "Faithfulness": round(float(faithfulness),2),
            "Answer Relevance": round(float(relevance),2),
            "BLEU Score": round(float(bleu),2),
```